



# Lambda vs Kappa Architecture

UrbanPulse — MetroConnect City Intelligence Platform · DSE ZG556

## Architecture Choice: Lambda (Hybrid) — Justified

UrbanPulse adopts a **Lambda Architecture** with **Apache Flink** as the speed layer and **Apache Spark** as the batch layer, sharing a single Apache Kafka ingestion bus.

**Why not pure Kappa?** The government reporting mandate is the deciding factor. MetroConnect must submit weekly ward energy reports to elected councillors and monthly AQI trend reports to the state government — both require **deterministic, reproducible, auditable batch results** stored as immutable Parquet files. A streaming-only Kappa approach cannot guarantee audit reproducibility when Kafka topics are compacted after 365 days, and historical reprocessing on budget-constrained city servers would starve the live alerting pipeline.

**Why Lambda works here?** The added complexity (two processing engines) is bounded and manageable: both Flink and Spark run as simple local-mode jobs on a single city server, all code is Python (no Java), all software is open-source (zero licence cost), and the batch jobs run off-peak so they don't compete with Flink for CPU during peak traffic hours.

### ⚡ Lambda

Separate speed + batch layers

### 🔄 Kappa

Single streaming layer only



#### Latency

AQI alert target: < 2 min  
Signal adapt: < 90 s

✓ WINS

#### Flink speed layer hits <30 s for AQI alerts

The **Flink speed layer** processes the 60 events/sec air quality stream and emits AQI>300 alerts in under 30 seconds — well inside the 2-minute target. Traffic signal adaptation fires within 90 s of gridlock detection. Batch layer (Spark) runs every 15 min for ward reports — acceptable for councillors, not for emergency response.

≈ COMPARABLE

#### Flink-on-Kappa also achieves sub-2-min alerts

A pure Kappa stack using Flink can match Lambda latency for real-time alerts. However, **historical reprocessing** (re-running 90 days of AQI data for a pollution audit) requires replaying the entire Kafka topic — slower than a pre-built batch job and more resource-intensive on city servers.



#### Fault Tolerance

RPO < 15 min  
RTO < 30 min

✓ WINS

#### Batch layer provides natural fallback

If the Flink speed layer crashes, the batch Spark job still produces 15-min ward summaries — councillor dashboards stay live. Kafka's **3-broker replication (RF=3)** means no data loss even if one broker fails (RPO = 0 for Kafka). Spark job can cold-restart from Kafka offset checkpoint — RTO < 15 min in testing.

✗ WEAKER

#### Single pipeline — one failure affects all outputs

In Kappa, if the single Flink pipeline fails, **both real-time alerts AND dashboard data** stop simultaneously. Recovery requires replaying from the saved Kafka offset — during replay, the signal control interface receives no updates. For a city safety system, this single point of failure is too risky.



#### Operational Complexity

City IT team size: small  
Open-source mandate

✗ HIGHER

#### Two processing systems to deploy and monitor

MetroConnect's IT team must maintain **both Flink and Spark**, plus their separate job schedulers, checkpointing paths, and monitoring. However, complexity is bounded — we use Docker Compose locally, PyFlink + PySpark (no Java needed), and a shared Kafka bus. The trade-off is accepted because of the batch reporting mandate.

✓ SIMPLER

#### Single streaming engine, one code base

Kappa uses **one Flink job** for everything — simpler CI/CD, fewer moving parts, easier for a small city IT team. No separate batch scheduler or Parquet partition management. However, "simpler" breaks down when councillor reports need historical re-runs on 365 days of smart meter data — Flink stateful reprocessing at that scale is not simple.



#### Reprocessing Capability

365-day smart meter audit  
AQI historical analysis

✓ WINS

#### Spark batch job re-runs cleanly on Parquet

When a **new ward boundary is drawn** (common in Indian city administration), the Spark batch job re-reads historical Parquet files and regenerates all ward summaries. Smart meter data retained 365 days in Kafka; Parquet archive kept indefinitely. Re-run time for 1 month of ward data: ~4 min on a single server.

✗ PAINFUL

#### Requires full Kafka replay — slow and resource-heavy

Kappa reprocessing means replaying **365 days × 1,100 events/sec** of smart meter data through the streaming job. At that volume, a full replay on city-owned servers (no cloud burst) takes hours and blocks the live pipeline. Kafka topic retention must match the longest audit window — 365 days of smart meter data is ~95 GB of compressed logs per broker.



#### Cost

≈ COMPARABLE

Slightly higher compute — Flink + Spark run concurrently

≈ COMPARABLE

Lower compute during normal ops; higher during reprocessing

|                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
|                                                                                                                                                                                                                                               | <div><b>Lambda</b></div> <div>Separate speed + batch layers</div>                                                                                                                                                                                                                                                                                                                                                                                                                                             | <div><b>Kappa</b></div> <div>Single streaming layer only</div>                                                                                                                                                                                                                                                                                                                                                                                                                           |                                           |
| Budget-constrained<br>On-premise servers only                                                                                                                                                                                                 | Running both Flink (speed layer) and Spark (batch layer) requires more RAM simultaneously — roughly <b>8–12 GB total</b> on the city server. However, Spark batch jobs run off-peak (2 AM) so peak-hour load is only Flink + Kafka. All software is open-source — zero licensing cost, satisfying the government open-source mandate.                                                                                                                                                                                                                                                          | Kappa needs only Flink + Kafka day-to-day — ~6 GB RAM. But during a historical reprocessing event (e.g. annual energy audit), it spikes to use <b>all available RAM</b> while replaying 95 GB of Kafka logs. On a budget-constrained city server with no elastic compute, this spike can starve the live alerting pipeline.                                                                                                                                                                                                                                                 |                                           |
| <div></div> <div><b>Compliance — Govt Reporting Mandate</b></div> <div>Weekly councillor reports<br/>Monthly state submissions<br/>Annual energy audit</div> | <div> <b>WINS</b></div> <div><b>Batch layer natively produces auditable, reproducible reports</b></div> <div>The Spark batch layer reads from <b>immutable Parquet files</b> partitioned by ward_id and date. Each run is deterministic — the same input always produces the same councillor report. State government auditors can independently re-run the Spark job on archived data. Weekly ward energy summaries and monthly AQI trend reports are generated automatically as scheduled Spark jobs.</div> | <div> <b>WEAKER</b></div> <div><b>Streaming outputs are harder to audit and reproduce</b></div> <div>Kappa streaming results (e.g. ward kWh totals) depend on the exact Kafka offset range replayed. If Kafka topic data is compacted or expired, <b>historical reports cannot be reproduced</b> — a compliance risk for state government submissions. Audit trail requires keeping 365 days of raw Kafka data per topic, which is expensive on-premise and not standard practice.</div> |                                           |
|                                                                                                                                                                                                                                               | <div><b>4</b></div> <div>Lambda Wins</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <div><b>1</b></div> <div>Kappa Wins</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <div><b>2</b></div> <div>Comparable</div> |